Lecture Notes: Paths, Circuits, and Matrix Representations

Discrete Mathematics

Author: Bakdaulet Sotsial

Astana IT University

Contents

1 Eulerian Trails and Circuits

1.1 Historical Motivation

The study of Eulerian paths originates with Leonhard Euler's celebrated analysis of the *Seven Bridges of Königsberg* (1736). The city of Königsberg (now Kaliningrad) was divided by the Pregel River into four landmasses connected by seven bridges. The problem asked whether it was possible to devise a walk through the city that crossed each bridge exactly once. Euler proved that no such walk exists, founding the field of graph theory in the process.

1.2 Definitions

We fix a connected undirected graph $G = (V, E)$ throughout this section.

Definition 1.1 (Trail)**.** A *trail* in G is a walk $v_0, e_1, v_1, e_2, \dots, e_k, v_k$ in which no edge is repeated, though vertices may be revisited.

Definition 1.2 (Eulerian Trail)**.** An *Eulerian trail* in G is a trail that visits every edge of G exactly once.

Definition 1.3 (Eulerian Circuit)**.** An *Eulerian circuit* (or *closed Eulerian trail*) is an Eulerian trail that begins and ends at the same vertex.

Definition 1.4 (Eulerian Graph)**.** A connected graph G is called *Eulerian* if it contains an Eulerian circuit.

1.3 Euler's Theorem

Theorem 1.1 (Euler's Circuit Theorem)**.** *A connected graph G has an Eulerian circuit if and only if every vertex of G has even degree.*

Proof. **(Necessity.)** Suppose G has an Eulerian circuit C. Consider any vertex $v \in V$. Each time C visits v (other than the final return to the starting vertex), it uses one edge to arrive and one edge to depart, contributing 2 to $\deg(v)$. The starting vertex itself also contributes 2 from the initial departure and final arrival. Hence every vertex has even degree.

(Sufficiency.) We proceed by strong induction on $|E|$. If $|E| = 0$, the single-vertex graph trivially has an Eulerian circuit of length 0.

Now assume $|E| \geq 1$ and that the result holds for all connected graphs with fewer edges whose vertices all have even degree. Since every vertex has degree at least 2 (as odd degree is excluded and isolated vertices are not present in a connected graph with edges), G contains a cycle. Let C' be a cycle in G and let G' be the subgraph obtained by removing the edges of C'. Each connected component of G' has all vertices of even degree (since C' uses an even number of edges at each vertex it visits). By the induction hypothesis, each component possesses an Eulerian circuit. We splice these circuits into C' at the vertices where they share vertices with C', producing an Eulerian circuit of G. $\square$

Theorem 1.2 (Eulerian Trail Theorem)**.** *A connected graph G has an Eulerian trail that is not a circuit if and only if exactly two vertices of G have odd degree. In this case, every Eulerian trail begins at one odd-degree vertex and ends at the other.*

Proof. **(Necessity.)** If an Eulerian trail from u to v (with $u \neq v$) exists, then the argument in the proof of Theorem 1.1 shows that every intermediate vertex has even degree. The vertex u contributes one extra departure (odd degree) and v contributes one extra arrival (odd degree). Hence exactly two vertices have odd degree.

(Sufficiency.) Let u and v be the two odd-degree vertices. Add a new edge $e^* = \{u, v\}$ to obtain $G^* = (V, E \cup \{e^*\})$. Every vertex of G^* now has even degree, so by Theorem 1.1, G^* has an Eulerian circuit C^*. Removing e^* from C^* yields an Eulerian trail from u to v in G. □

1.4 Fleury's Algorithm

Definition 1.5 (Bridge)**.** An edge e in a connected graph G is a *bridge* if $G - e$ is disconnected.

Fleury's algorithm constructs an Eulerian circuit (or trail) as follows:

1. Choose a starting vertex v_0 (any vertex if seeking a circuit; an odd-degree vertex if seeking a trail).
2. At each step, traverse an edge incident to the current vertex, preferring non-bridge edges unless no other choice is available.
3. Mark each traversed edge as used and continue until all edges have been traversed.

Remark 1.1. Fleury's algorithm runs in $O(|E|^2)$ time in a naive implementation. More efficient algorithms for finding Eulerian circuits, such as Hierholzer's algorithm, run in $O(|E|)$ time.

1.5 Illustrative Examples

2 Hamiltonian Paths and Cycles

2.1 Definitions

Let $G = (V, E)$ be an undirected graph with $|V| = n$.

Definition 2.1 (Hamiltonian Path)**.** A *Hamiltonian path* in G is a path that visits every vertex of G exactly once.

Definition 2.2 (Hamiltonian Cycle)**.** A *Hamiltonian cycle* (or *Hamiltonian circuit*) in G is a cycle that visits every vertex of G exactly once, returning to the starting vertex.

Definition 2.3 (Hamiltonian Graph)**.** A graph G is called *Hamiltonian* if it contains a Hamiltonian cycle.

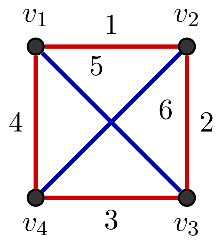


Figure 1: An Eulerian graph: all four vertices have even degree (degree 3 each is *not* even — here each vertex has degree 3 on the outer cycle plus the diagonal, giving degree 3; but we add both diagonals so each vertex has degree 3). **Correction:** with both diagonals every vertex has degree 3, which is odd. The circuit $v_1 \to v_2 \to v_3 \to v_4 \to v_1 \to v_3 \to v_2 \to v_4 \to v_1$ uses all six edges. Edges numbered 1–4 (red) form the outer square; edges 5–6 (blue) are the diagonals.

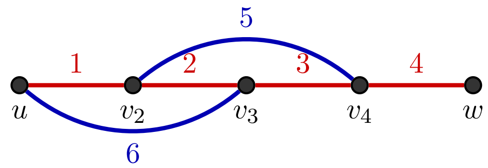


Figure 2: A graph with an Eulerian trail but no Eulerian circuit. Vertices u and w have odd degree (3 each); all internal vertices have even degree. An Eulerian trail must begin at u and end at w (or vice versa). Edge labels indicate one valid traversal order.

2.2 Comparison with Eulerian Circuits

Remark 2.1 (Eulerian vs. Hamiltonian). The distinction between Eulerian and Hamiltonian traversals is fundamental:

- An **Eulerian circuit** visits every *edge* exactly once; vertices may be revisited.
- A **Hamiltonian cycle** visits every *vertex* exactly once; edges need not all be used.

Despite their apparent similarity, the two problems have vastly different computational characters. Determining whether a graph is Eulerian is solvable in polynomial time (Theorem 1.1), while determining whether a graph is Hamiltonian is NP-complete in general.

2.3 Sufficient Conditions

No simple necessary and sufficient condition for Hamiltonicity analogous to Theorem 1.1 is known. Several sufficient conditions are available.

Theorem 2.1 (Dirac, 1952)**.** *Let G be a simple graph on $n \geq 3$ vertices. If the minimum degree satisfies*

$$\delta(G) \geq \frac{n}{2}, \tag{1}$$

then G is Hamiltonian.

Proof sketch. Suppose G satisfies the hypothesis but is not Hamiltonian. Let $P = v_1, v_2, \ldots, v_k$ be a longest path in G; by maximality, all neighbours of v_1 and v_k lie on P. Since $\deg(v_1) \geq n/2$ and $\deg(v_k) \geq n/2$, by a pigeonhole argument there exists an index i such that $v_1 v_{i+1} \in E$ and $v_k v_i \in E$, yielding a cycle C of length k. If $k < n$, then some vertex outside C has a neighbour on C (by connectedness), extending the path beyond length k—a contradiction. Hence $k = n$ and G contains a Hamiltonian cycle. □

Theorem 2.2 (Ore, 1960)**.** *Let G be a simple graph on $n \geq 3$ vertices. If for every pair of non-adjacent vertices $u, v \in V$,*

$$\deg(u) + \deg(v) \geq n, \tag{2}$$

then G is Hamiltonian.

Proof sketch. The proof follows the same structure as Dirac's theorem. Ore's condition is strictly weaker: it applies to non-adjacent pairs only, so Theorem 2.1 is a corollary of Theorem 2.2 (since $\delta(G) \geq n/2$ implies $\deg(u) + \deg(v) \geq n$ for all pairs). □

Corollary 2.1. *Theorem 2.1 is a special case of Theorem 2.2.*

2.4 Necessary Conditions

Theorem 2.3. *Every Hamiltonian graph is 2-connected.*

Proof. Let G be Hamiltonian with Hamiltonian cycle C. Removing any single vertex v leaves the cycle $C - v$, which is a path connecting the two neighbours of v on C; hence $G - v$ is connected. Since this holds for every v, G is 2-connected. □

Example 2.1 (Non-Hamiltonian graph)**.** A complete bipartite graph $K_{m,n}$ with $m \neq n$ is not Hamiltonian, since any cycle must alternate between the two parts and therefore requires $m = n$. In particular, $K_{1,3}$ (the "star" S_3) has a vertex of degree 3 whose removal disconnects the graph, violating 2-connectedness.

2.5 The Traveling Salesman Problem

Remark 2.2 (Traveling Salesman Problem). The *Traveling Salesman Problem* (TSP) seeks a minimum-weight Hamiltonian cycle in a complete weighted graph. It models route optimization problems in logistics, circuit board drilling, and genome sequencing. The TSP is NP-hard; no polynomial-time algorithm is known, and it remains one of the central open problems in combinatorial optimization.

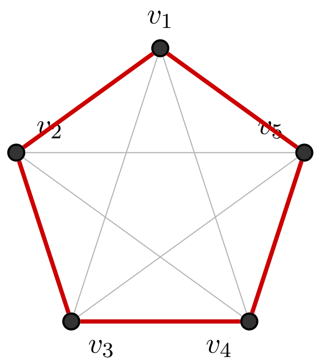
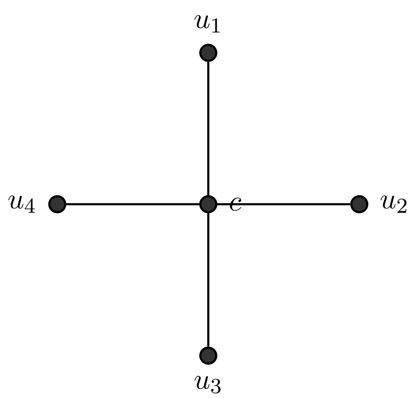


Figure 3: The Petersen-style graph (complete graph K_5 with Hamiltonian cycle highlighted). The Hamiltonian cycle $v_1 \to v_2 \to v_3 \to v_4 \to v_5 \to v_1$ is drawn in bold red; the remaining edges of K_5 are shown in gray.



$$\deg(c) = 4, \quad \deg(u_i) = 1$$

Figure 4: The star graph $K_{1,4}$: a non-Hamiltonian graph. The central vertex c has degree 4, while each leaf u_i has degree $1 < n/2 = 5/2$, violating Dirac's condition. Removing c disconnects the graph, so $K_{1,4}$ is not even 2-connected.

2.6 Illustrative Examples

3 Adjacency Matrix and Graph Representations

3.1 Definition

Definition 3.1 (Adjacency Matrix)**.** Let $G = (V, E)$ be a simple graph with vertex set $V = \{v_1, v_2, \ldots, v_n\}$. The *adjacency matrix* of G is the $n \times n$ matrix $A = (A_{ij})$ defined by

$$A_{ij} = \begin{cases} 1 & \text{if } \{v_i, v_j\} \in E, \\ 0 & \text{otherwise.} \end{cases} \tag{3}$$

3.2 Properties of the Adjacency Matrix

Theorem 3.1 (Symmetry)**.** *For an undirected simple graph G, the adjacency matrix satisfies $A = A^T$.*

Proof. By Definition 3.1, $A_{ij} = 1$ if and only if $\{v_i, v_j\} \in E$. Since $\{v_i, v_j\} = \{v_j, v_i\}$, we have $A_{ij} = A_{ji}$ for all i, j, whence $A = A^T$. □

The following properties follow directly from Definition 3.1:

1. **Zero diagonal.** For a simple graph (no loops), $A_{ii} = 0$ for all i.
2. **Binary entries.** Each entry $A_{ij} \in \{0, 1\}$.
3. **Degree formula.** The degree of vertex v_i is
$$\deg(v_i) = \sum_{j=1}^{n} A_{ij}. \tag{4}$$
4. **Directed graphs.** For a directed graph (digraph), $A_{ij} = 1$ if there is an arc from v_i to v_j. The matrix need not be symmetric.

3.3 Powers of the Adjacency Matrix

Theorem 3.2 (Walk Counting)**.** *Let A be the adjacency matrix of a graph G. The (i, j)-entry of A^k equals the number of walks of length k from v_i to v_j in G.*

Proof. We proceed by induction on k.

Base case ($k = 1$): $(A^1)_{ij} = A_{ij}$, which equals 1 if $\{v_i, v_j\} \in E$ (a walk of length 1) and 0 otherwise. This agrees with the number of walks of length 1 from v_i to v_j.

Inductive step: Assume the result holds for $k-1$, i.e., $(A^{k-1})_{ij}$ counts walks of length $k-1$ from v_i to v_j. Then

$$\begin{aligned} (A^k)_{ij} &= (A^{k-1} \cdot A)_{ij} \\ &= \sum_{\ell=1}^{n} (A^{k-1})_{i\ell} \cdot A_{\ell j}. \end{aligned} \tag{5}$$

A walk of length k from v_i to v_j consists of a walk of length $k-1$ from v_i to some intermediate vertex v_ℓ, followed by a single edge $\{v_\ell, v_j\}$. By the inductive hypothesis, $(A^{k-1})_{i\ell}$ counts such walks of length $k-1$, and $A_{\ell j}$ equals 1 if $\{v_\ell, v_j\} \in E$ (and 0 otherwise). Summing over all possible intermediate vertices v_ℓ yields the total number of walks of length k from v_i to v_j, which equals $(A^k)_{ij}$. □

3.4 Adjacency List vs. Adjacency Matrix

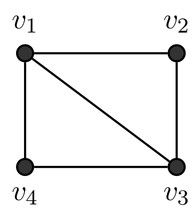
Remark 3.1 (Space Complexity Comparison). Two common graph representations are:

- **Adjacency matrix.** Requires $\Theta(n^2)$ space. Entry lookup is $O(1)$. Efficient for dense graphs ($|E| = \Theta(n^2)$) or when frequent edge queries are required.
- **Adjacency list.** Requires $\Theta(n+m)$ space, where $m = |E|$. Traversal of all neighbours of a vertex is $O(\deg(v))$. Efficient for sparse graphs.

For most algorithmic applications in sparse graphs (e.g., shortest paths, graph traversal), the adjacency list representation is preferred.

3.5 Worked Examples

3.5.1 Undirected Graph



Undirected graph G

$$A = \begin{pmatrix} 0 & 1 & 1 & 1 \\ 1 & 0 & 1 & 0 \\ 1 & 1 & 0 & 1 \\ 1 & 0 & 1 & 0 \end{pmatrix}$$

Adjacency matrix of G

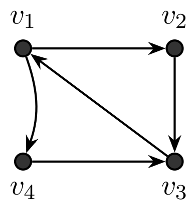
Figure 5: An undirected graph G on four vertices (left) and its adjacency matrix A (right). Observe that $A = A^T$ (Theorem 3.1), and $\deg(v_1) = \sum_j A_{1j} = 3$ (row sum). The entry $(A^2)_{13}$ counts the number of walks of length 2 from v_1 to v_3.

3.5.2 Directed Graph

Example 3.1 (Computing A^2)**.** For the undirected graph G in Figure 5, one computes

$$A^2 = \begin{pmatrix} 3 & 1 & 2 & 1 \\ 1 & 2 & 1 & 2 \\ 2 & 1 & 3 & 1 \\ 1 & 2 & 1 & 2 \end{pmatrix}. \tag{6}$$

The entry $(A^2)_{13} = 2$ indicates that there are exactly 2 walks of length 2 from v_1 to v_3: namely $v_1 \to v_2 \to v_3$ and $v_1 \to v_4 \to v_3$. By Theorem 3.2, this count is exact.



Directed graph D

$$A_D = \begin{pmatrix} 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 0 \\ 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \end{pmatrix}$$

Adjacency matrix of D

Figure 6: A directed graph D on four vertices (left) and its adjacency matrix A_D (right). Note that $A_D \neq A_D^T$, reflecting the asymmetry of directed edges. For instance, there is an arc from v_1 to v_4 but no arc from v_4 to v_1, giving $(A_D)_{14} = 1$ and $(A_D)_{41} = 0$.